

Docker: A Favourite in the DevOps World

Docker is the world's leading containerisation platform designed to make it easier to create, deploy and run various applications by using containers. Docker containers wrap up a piece of software into a complete file system that contains everything the software needs to run.



Have you ever developed a snippet of code in Java on your local system and then tried to run it on some other desktop with a different environment? You were probably blocked with a configuration error or exception. If the snippet of code works fine on your system but doesn't on the system with a different configuration, then you need to make changes to the latter's configuration in order to make the code run. Sometimes, this gets quite tedious and infuriating.

Containers actually put a full stop to all such problems. They help us run any software reliably when moved from one computing environment to another, whether from a developer's desktop to a test environment or into production, and perhaps even from a physical machine in a data centre to any virtual machine in a public or private cloud. The real need for software containers arises when the supporting environment on which the software is actually developed is not identical to the one where it's implemented. For instance, if you test code using Python 2.7 and then it's made to run on Python 3 in production, something weird

happens. There are cases when we rely on the behaviour of a certain version of a library and another updated version gets installed automatically, resulting in the software running on that system behaving unusually. Not only different software, configurations or a library, but even a different network topology or different security policies and storage can prevent software from running successfully.

A container basically consists of an entire runtime environment, which includes an application with all its dependencies, like libraries and other binaries, and even the configuration files needed to run it, all bundled into one package. Hence, by containerising the differences in operating system distributions, the underlying infrastructure is abstracted away. Containers encapsulate all the discrete components of application logic which are provisioned, that too with the minimal resources needed to do their job.

Containers are different from virtual machines

Unlike virtual machines (VMs), containers have no need for embedded operating systems; calls are made for operating